CODING AGENTS AND REVIEW

The review loop



How to check what an agent wrote, and make every review better than the last.

Kenneth Rohde Christiansen

review-loop · open source, Apache-2.0

OPENING

The agent writes the change in minutes. Someone still has to be sure it is right.

This talk is about that someone, and how to stop them being a bottleneck.

OPENING

Where we are going

01

The problem

Writing got cheap. Reading did not.

02

Why it matters

What one team found, and how review-loop answers it.

03

The loop

What a review is made of, and what it costs.

04

In practice

Four levels, and learning from outside review.

05

For managers

What to measure, who owns it, how to start.

The problem

Writing got cheap. Reading did not.

ACT I · THE PROBLEM

A change you did not type is a change you have to read.

And an agent produces them faster than anyone can read.

ACT I · THE PROBLEM

Two moments to find out a change is wrong

After it goes out

- A reviewer, a bot or a user finds it
- You fix it in another round
- The next change gets the same comment

Before it goes out

- The same checks run first
- It is fixed in the same round
- The class of bug gets a check

8 of 9

outside review comments on three pull requests were right.

> **All eight were fixed. None of them changed the next review.** The fixes disappeared into the pull requests.

Our own: a real round on one of our projects, from the review-loop README. Copilot comments on three pull requests.

ACT I · THE PROBLEM

Fixing a comment fixes one line. The next change gets the same comment.

Nothing in the process remembers.

ACT I · THE PROBLEM

Most of our time goes on reading, and how we read is almost never discussed.

So it is never improved.

Tudor Girba and Simon Wardley, "Rewilding software engineering", chapter 1, 2025.

ACT I · THE PROBLEM

The post this talk keeps coming back to

How the Claude Code team does code review

Addy Osmani, Morgan Westlee Lunt and Qing Wang

claude.dev blog · 15 October 2026 · a 14 minute read

`claude.dev/blog/solving-code-review-at-anthropic`

Scan to read it

Osmani, Lunt and Wang, claude.dev, 15 October 2026.

ACT I · THE PROBLEM

The Claude Code team reviews its agent-written code in layers.

Verify before the pull request. An AI review on every pull request, its findings checked by other agents. A file that says what matters. Past incidents as the test set.

Osmani, Lunt and Wang, "How the Claude Code team does code review", claude.dev, 15 October 2026.

ACT I · THE PROBLEM

Several of these points are also our own conclusions, from working on our own products.

We hit the same problems: comments that taught the next review nothing, one reader finding only part of what outside reviewers found, three readers costing three times one. review-loop is what we built.

The numbers shown for review-loop are our own measurements, small and early.

ACT I · THE PROBLEM

Not a step at the end. A loop that gets better each round.

Check it yourself. Let others check it. Turn what they find into a check that runs.

Why it matters

What one team found when agents wrote most of the code, and how review-loop answers it.

30,000

changes a day landing in Anthropic's monorepo in September, up from about 2,500 in May.

Claude writes about 90% of Claude Code's own code. Claude Code has its own repository, and the post says it feels the same pressure.

Osmani, Lunt and Wang, "How the Claude Code team does code review", claude.dev, 15 October 2026. The 30,000 is for the monorepo, not for Claude Code's repository.

When you ship this fast, line-by-line review is incredibly difficult, if not impossible.

They did not drop review. They moved most of the reading to machines and kept people for what machines cannot decide.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

The goal: less human time per change, and no more reverts than before.

Reverts and incidents are the guardrail. Human approvals per pull request, human comments and the wait for a person are what they push down.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

40%

drop in the share of merged pull requests that had to be reverted, while the number merged more than doubled.

Claude Code repositories, June to September. The post gives the direction and the share, not the counts.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

54%

of internal pull requests got substantive review comments, up from 16%.

> **At the launch of the research preview of their AI review.** The post does not say whether 54% is the current figure.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

ACT II · WHY IT MATTERS

What the post does, and where it lives here

THE POST	IN REVIEW-LOOP
Verify before the pull request, with a fresh reader	`/pre-pr-review`
A REVIEW.md written by the code owners	`REVIEW.md`, created by `review-loop init`
Parallel reviewers, a vote, findings tagged Important, Nit or Pre-existing	`multipass`: three readers, a vote, the same labels
Every finding fixed, or resolved with a reason	`review-loop gate`: a report with Open: 0

Practices from Osmani, Lunt and Wang, 15 October 2026. The right column is review-loop's own README.

ACT II · WHY IT MATTERS

...and the rest of the map

THE POST	IN REVIEW-LOOP
Fix recurring findings at the source	Sweeps in `REVIEW.sweeps`, and `/retro`
Review has to converge	Three cycles, only the new diff, fix the class
Past incidents as the reviewer's test set, noise measured	`/review-eval`: recall and noise
Merge queue, feature flags, gradual release	Not covered. Risk tiers: planned

Practices from Osmani, Lunt and Wang, 15 October 2026. The right column is review-loop's own README.

ACT II · WHY IT MATTERS

Review has to converge

INTERNAL PULL REQUESTS	BEFORE, THEN AFTER
Pass after one review-and-fix round	65%, then 78%
Need five or more rounds	8.9%, then 1.5%
Mean number of reviews to pass	2.08, then 1.32

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026. Before and after the team's convergence work.

One pull request went through more than fifty rounds of review and fixes overnight.

A person dismisses a nit in seconds. An agent makes another commit and starts another round. review-loop stops at three cycles, reads only the new diff, and fixes the class of a bug, not the line.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

ACT II · WHY IT MATTERS

59%

of what the reviewer kept flagging was old code, there since the first pass.

review-loop marks it Pre-existing and keeps it out of the author's count, so a round does not grow on code the change never touched.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026. By git blame, on the pull requests the team studied.

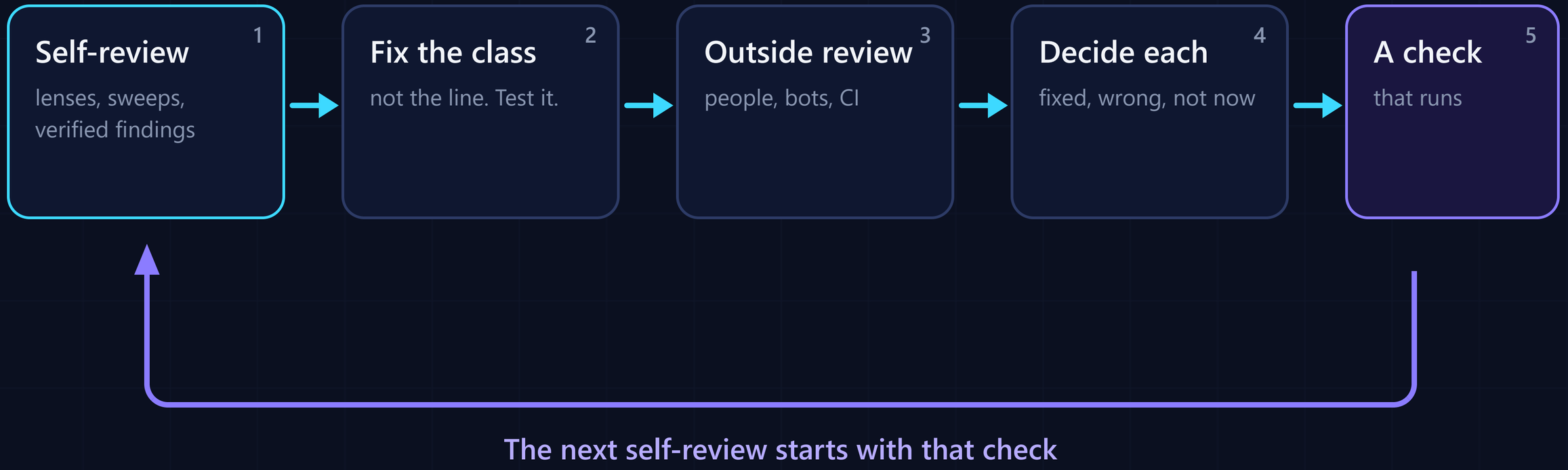


The loop

What one review is made of, and what it costs.

ACT III · THE LOOP

Five parts, one loop



ACT III · THE LOOP

A good reviewer reads whole functions, not diff lines.

The callers and siblings needed to answer a question. The report says what was read in full and what was only skimmed.

ACT III · THE LOOP

Nineteen lenses. One question each.

Untrusted input

Missing, null, the wrong type, huge, negative, NaN?

Error paths

Can it report success after the write failed?

State

Every writer of a flag, and every reset of it.

Claims

A comment says "never". Is that true for every caller?

skills/pre-pr-review/SKILL.md, section 3. Others: limits, cost, routing, platform, tests, concurrency, UI, dead code, other doors to the same resource.

ACT III · THE LOOP

A small change, three problems

```
function handle(body, settings) {
  const req = JSON.parse(body);
  const n = parseInt(req.limit);
  const items = req.tags.map((t) => t.trim());
  settings.lastLimit = n;
  return { ok: true, count: items.length, limit: n };
}

function showTitle(el, title) {
  el.innerHTML = title;
}
```

Twelve lines. Read it through the lenses and ask, for each line, what input breaks it.

A small example change made for this talk.

ACT III · THE LOOP

Verify: write the failure, then run it

```
$ node probe.js
{"limit":"1.5","tags":[" a "]} -> limit 1
{"limit":"abc","tags":[]}      -> limit null
{"limit":"5"}                  -> TypeError: reading 'map'
null                           -> TypeError: reading 'limit'
```

Every finding names the input that breaks it. What cannot be made concrete is dropped, or listed as "could not confirm".

Output of the probe on the example change, Node 22.

ACT III · THE LOOP

Outside reviewers are cheap and sometimes wrong. Yours should not be.

A finding without a concrete failure does not go in the report.

37%

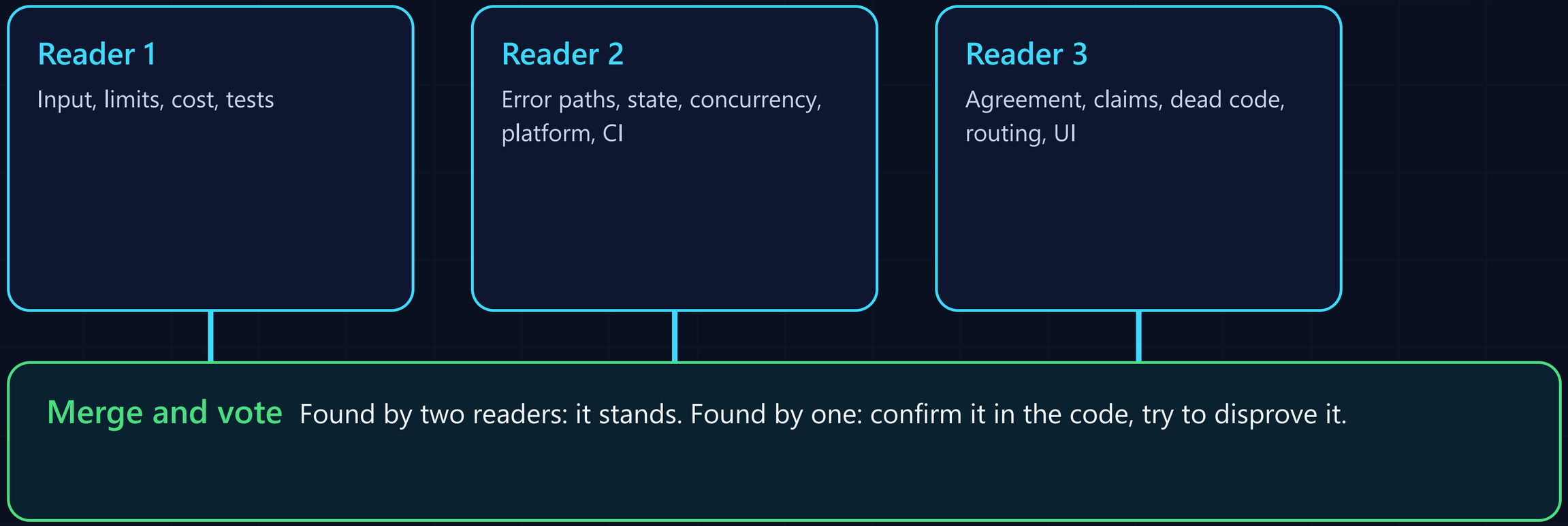
of the real problems found by one cheap reader.

> **Our first baseline, code only, the cheapest model for every reader.** A small set of cases. It is the reason for the next slide, not a general result.

review-loop's own first baseline, in skills/pre-pr-review/references/multipass.md.

ACT III · THE LOOP

Three readers, each with one job, then a vote



A fourth reader checks the change against an external specification, when REVIEW.md lists one. Readers are not told what the others look for.

ACT III · THE LOOP

Pay for three readers only when it matters

One reader

- Documentation and tests
- A small code change, away from any guard
- Costs one read of the diff

Three readers and a vote

- A larger change
- Always, for a path marked as a guard
- Costs about three reads

review-loop depth, skills/pre-pr-review/scripts/review-depth.sh. Default limit: 60 changed code lines.

ACT III · THE LOOP

It asks before you spend

```
$ review-loop depth HEAD~1 HEAD
depth: single
why: small: 10 code lines (limit 60)

$ review-loop depth HEAD~7 HEAD
depth: multipass
why: 393 code lines (a small change is up to 60)

$ review-loop depth HEAD~20 HEAD
depth: multipass
why: 1731 code lines (a small change is up to 60)
```

The last commit, the last seven and the last twenty of a real game.

Run on Neon Smash (github.com/kenchris/soma-breakout), October 2026.

ACT III · THE LOOP

Three cycles at most. Then ask whether the approach is wrong.

After the first pass, read only the new diff. Fix the class of a bug, not the line.

ACT III · THE LOOP

What failure means here goes in one short file.

REVIEW.md: what counts as a failure, which paths guard something, what to ignore, which classes were raised before. The reviewer reads it first.

IV

Checks that run

Attention is expensive. A search is free.

ACT IV · CHECKS THAT RUN

If a search can find the pattern, do not ask a reader to remember it.

A line in a prompt costs attention on every review. A sweep costs a second and no model.

ACT IV · CHECKS THAT RUN

Sweeps: the added lines, run through search rules

```
$ review-loop sweeps HEAD~20 HEAD
[json-parse] throws on bad input: is the parsed
  shape checked before use?
  js/assist.js:38:  JSON.parse(localStorage.getItem(...))
[new-listener] is there a matching removal?
  js/secretCode.js:191:  document.addEventListener(...)
diff-sweeps: 15 question(s) for the author (warn-only)
```

No model, no cost. Each hit is a question.

Run on Neon Smash (github.com/kenchris/soma-breakout), last 20 commits, output shortened.

ACT IV · CHECKS THAT RUN

A hit is a question, not a finding.

Both of these are fine. The parse sits in a try and clamps the value. The listeners are added once, in a page that lives as long as the game. Answering takes a second.

js/assist.js and js/secretCode.js in Neon Smash.

RECORDED RUN

Depth and sweeps on a real game

RECORDED RUN



Last commit: one reader. Last seven: three. Then the added lines run through the sweeps: two questions, no model.

Windows 11, Git Bash. The output is real; the typing is replayed. Neon Smash, github.com/kenchris/soma-breakout

ACT IV · CHECKS THAT RUN

Your own sweep is one line

```
unchecked-parseint ## *.js ## parseInt\([^,)]*\) ## no radix, no NaN check:
  what do "abc" and "1.5" give?
```

name ## which files ## regex ## why. It goes in REVIEW.sweeps at the root of the repository.

The format of REVIEW.sweeps.

ACT IV · CHECKS THAT RUN

The gate: "reviewed" becomes a rule

```
$ review-loop gate main HEAD
FAIL: 1 commit(s) are in no review report
  with Open: 0, for example:
  b23e6f4 Add request handler
STOP: run the multipass review and fix
  or explain every finding

$ review-loop gate main HEAD~1
review gate: all 1 commit(s) covered by reports
```

A report in reviews/ with Open: 0 must cover every commit.

Run on the example change made for this talk.

ACT IV · CHECKS THAT RUN

The gate checks that a review was written. Not that it was good.

It is a hand-over rule. Quality comes from the lenses, the readers, the verification and the eval.

RECORDED RUN

The gate fails, then passes

RECORDED RUN



No report: it stops. A report with Open: 0 that covers the commit: it passes.

Windows 11, Git Bash. The output is real; the typing is replayed. The report was written by hand.

V

In practice

Four levels. Stop when it is enough.

ACT V · IN PRACTICE

Each level adds one thing

01

Ask for a review

One command, no setup.

02

Tell it about your repo

Two minutes, once.

03

Make "reviewed" a rule

For teams.

04

Learn from outside review

Optional, pays off after a few rounds.

ACT V · LEVEL 1

Ask for a review

```
/pre-pr-review
/pre-pr-review 123
/pre-pr-review main..my-branch
/pre-pr-review multipass
```

The current branch, a pull request, a range, or three readers and a vote. Fix what it finds and run it again.

These are typed to the agent (Claude Code, or VS Code), not to a shell.

ACT V · LEVEL 1

Install

```
git clone https://github.com/kenchris/review-loop
cd review-loop
./install.sh --link
review-loop doctor

# Windows
.\install.cmd --for all --add-to-path
```

Needs bash and git. On Windows, install.cmd finds the bash of Git for Windows.

README, Install.

ACT V · LEVEL 2

Tell it about your repository

```
$ review-loop init
created  ./REVIEW.md
created  ./REVIEW.sweeps
created  ./REVIEW.gate
ready    ./reviews/
created  ./.review-loop.env

$ review-loop doctor
8 ok, 0 missing
```

It never overwrites. Edit REVIEW.md a little: the reviewer reads it first.

Run on Neon Smash (github.com/kenchris/soma-breakout).

ACT V · LEVEL 2

A REVIEW.md for a small game

```
## What a failure is here
- Touch and keyboard doing different things
  for the same input.
- A rule CLAUDE.md describes and the code
  does not do.

## Guards: read in full
- js/secretCode.js  (once per game, never
  in a boss fight)
- sw.js  (a stale cache serves an old game)
```

Short on purpose. A reviewer that has to read a long file reads it badly.

REVIEW.md written for Neon Smash for this talk.

RECORDED RUN

Level 2 in ten seconds

RECORDED RUN



init creates REVIEW.md, REVIEW.sweeps, REVIEW.gate and reviews/. doctor checks the install.

Windows 11, Git Bash. The output is real; the typing is replayed. Neon Smash, github.com/kenchris/soma-breakout

RECORDED RUN

The same checks inside VS Code

RECORDED RUN



Terminal, Run Task, review-loop: sweeps. The wait for the terminal is cut. The hits are added as warnings on the lines, in the Problems panel and in the editor.

VS Code on Windows 11, a clean profile, Neon Smash. Recorded live: the task output and the warnings are real.

RECORDED RUN

/pre-pr-review in Copilot chat

RECORDED RUN



Agent mode in VS Code, on the last commit of Neon Smash. The report: no Important or Nit findings, and it says it could not run the tests.

VS Code on Windows 11, recorded live. Typing 2 times and the agent's work 6 times faster, still stretches cut. /pre-pr-review HEAD~1..HEAD, a ten-line change.

ACT V · LEVEL 2

Not every project wants this yet. One or two developers can try it first.

Keep REVIEW.md, the sweeps and the reports in a separate private repository. The skills are installed per developer, so the project itself does not change.

review-loop reads its settings from a file outside the repository when REVIEW_LOOP_CONFIG names one.

ACT V · LEVEL 2

The project stays as it is

The project

neon-smash, as it is

```
js/
tests/
sw.js
CLAUDE.md
no REVIEW.md
no reviews/
nothing to merge
```

reads its settings from



REVIEW_LOOP_CONFIG

review-loop-private

a separate repository, yours

```
neon-smash/  one folder per project
  config.env
  REVIEW.md
  REVIEW.sweeps
  reviews/
  evals/
```

Our idea: a review-loop-private repository with one folder per project.

ACT V · LEVEL 2

One file says where everything is

```
# review-loop-private/neon-smash/config.env
REVIEW_FILE=$HOME/review-loop-private/neon-smash/REVIEW.md
REVIEW_SWEEPS=$HOME/review-loop-private/neon-smash/REVIEW.sweeps
REVIEWS_DIR=$HOME/review-loop-private/neon-smash/reviews
EVALS_DIR=$HOME/review-loop-private/neon-smash/evals
```

The reviewer reads REVIEW.md from there, the sweeps run from there, the reports and eval cases are kept there.

review-loop help, Configuration.

ACT V · LEVEL 2

Same commands, nothing added to the project

```
$ export REVIEW_LOOP_CONFIG=~/review-loop-private/neon-smash/config.env
$ cd neon-smash
$ review-loop doctor
ok       REVIEW.md (~/review-loop-private/neon-smash/REVIEW.md)
ok       REVIEW.sweeps (optional)
8 ok, 0 missing

$ review-loop sweeps HEAD~7 HEAD
[timer-in-loop] a timer: is it cleared when the game is reset or paused?
  js/secretCode.js:63:   for (let i = 0; i < ticks; i++) setTimeout(...)

$ git status --short
$
```

A sweep from the private file fires. The project has no new files and nothing to commit.

Run on Neon Smash, HEAD~7 to HEAD, with the private files in a separate folder. Output shortened, paths shortened.

ACT V · LEVEL 2

What a private setup gives, and what it does not

Works the same

- /pre-pr-review, depth and sweeps
- Reports, scores and eval cases
- /retro on the outside comments you get

Does not, until it moves in

- CI and the gate cannot see files that are not in the repository
- Teammates review with their own settings
- Nothing is shared yet

Our reading of the configuration. Not tested in CI.

ACT V · LEVEL 2

Moving in later is a copy.

When the team says yes, copy the files to the project root and delete the config file. The root is the default place for all of them.

review-loop help: the defaults are REVIEW.md, REVIEW.sweeps, reviews/ and evals/ at the repository root.

ACT V · LEVEL 3

Make "reviewed" a rule

```
/pre-pr-review multipass my-branch
    -> reviews/feature-limits-b23e6f49.md
       Open: 0

review-loop gate main HEAD
review gate: all 1 commit(s) covered by reports
```

Commit the report. Check before a hand-over or merge.

Run on the example change made for this talk. The report is written by the agent, here by hand.

ACT V · LEVEL 3

In CI the scripts run. The skills do not.

Sweep hits appear as warnings on the changed lines of a pull request.
The gate can be a required check.

docs/ci.md. The sample workflow is a sketch, not yet run on GitHub.

ACT V · IN PRACTICE

Plain SKILL.md files. They work where an agent loads skills.

Claude Code, VS Code (Copilot agent mode and the Claude Code extension), other agents that read a skills folder. The scripts need only bash and git.

README, VS Code.

VI

Learning

Every outside comment is a check you did not have.

ACT VI · LEARNING

If a bot reviews every push, every push can bring new findings.

Copilot's review, CI and linters comment after you push. You fix, you push, and the next round finds something else. It starts to feel like a loop with no end. This part is for repositories where that is switched on.

Our experience. A real round: slide 7.

ACT VI · LEARNING

From a review thread to the next review

Threads open review comments on GitHub

`threads sync`

Ledger one row per thread, all undecided

`threads worksheet`

Your verdicts fixed, rejected, deferred

`threads verdict`

/retro sorts the misses by cause

Proposals

Add this sweep.
Add this line.
Delete that line.

Nothing changes until you say which to apply.

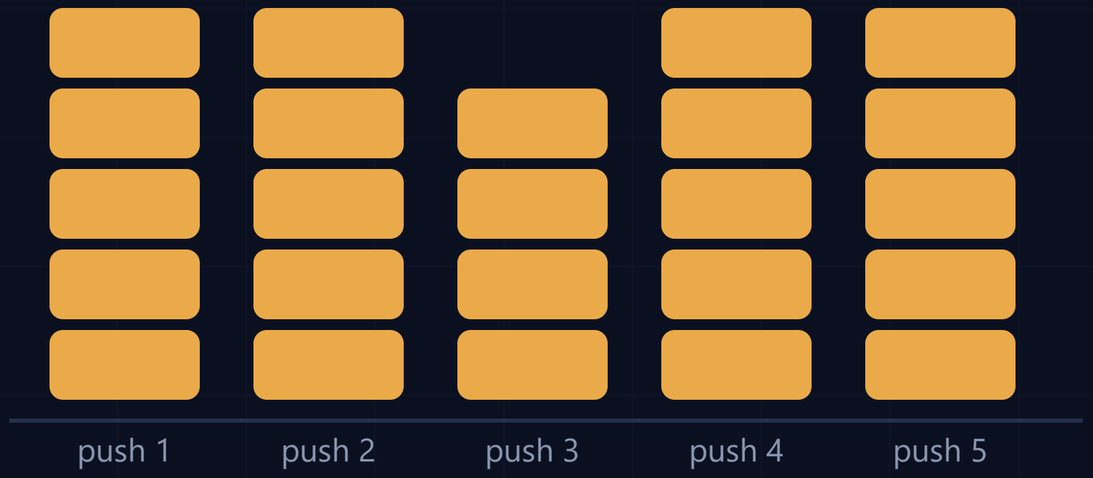
Every outside finding is a check you did not have. Learn from it, and the next push has fewer.

ACT VI · LEARNING

The goal: fewer findings after each push

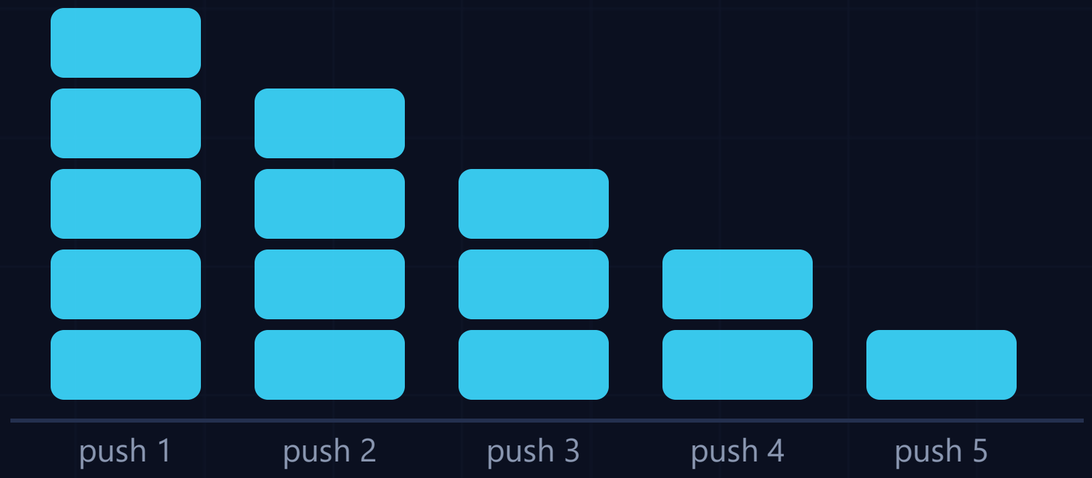
Fix, then push

every round finds something



Fix, learn, then push

each round leaves a check behind



Each square is a finding from a bot or a reviewer after a push. Illustration, not data. Measure your own: `review-loop score show` records, per round, the outside findings your own review had not reported.

An illustration of the aim. The number to watch is outside-new in review-loop score show.

ACT VI · LEARNING

Level 4: three commands

```
review-loop threads sync --repo owner/name

/triage

/retro round 26
```

Download the open threads. The agent proposes a verdict for each, and you confirm once. Then it asks what would have caught them. By hand: threads worksheet and threads verdict.

docs/learning-from-reviews.md. Needs node 18 and a token that can read the pull requests.

ACT VI · LEARNING

Deciding every thread by hand is the part people skip.

/triage reads each undecided thread against the code, proposes a verdict with the evidence, and records the ones you confirm in one call. Nothing is posted to GitHub and no code changes.

skills/triage/SKILL.md in review-loop.

ACT VI · LEARNING

What /triage puts in front of you

```
id       where                   who      proposal  evidence
a1b2c3d  js/secretCode.js:196    copilot  rejected  added once, at start-up
e4f5a6b  js/assist.js:38         copilot  rejected  parse is in a try, value clamped

Apply all, or tell me which ids to change.

review-loop threads verdict --batch decisions.tsv
```

One table, grouped by proposal, each with a line of evidence you can check. All or nothing when it records.

An example built from two sweep hits earlier in this talk, not real threads. The skill is tested at the script level, not yet on a real round.

ACT VI · LEARNING

A verdict is your decision about one thread

VERDICT	MEANING
fixed	Right, and fixed. Give the commit.
rejected	Wrong. Say why: it is remembered.
caught	My own review had already reported it.
deferred	Real, but not now.

docs/learning-from-reviews.md. Also: missed, and new for undecided.

ACT VI · LEARNING

The cheapest change that would have caught it

1. **A check that runs** — a sweep, a lint rule, a test, a CI job
2. **A pointer** — one line that says where to look
3. **A lens or reader** — a different way of reading
4. **Context** — the spec, the caller list, the config
5. **A REVIEW.md line** — last: only for a judgement call

Take the first that fits.
A hit costs a second.
A line costs attention on every review.

ACT VI · LEARNING

What /retro said about a real round

Three

The lens covered it, the reader skipped it.

One

A one-line search would find it: a pointer cast with no family check.

One

A deleted test file nobody had noticed.

Result

A ranked list of changes, instead of nine fixes that teach nothing.

The Copilot round from slide 7: nine comments, eight right. The README does not break down the other three.

ACT VI · LEARNING

Measure the reviewer too. Recall and noise. Both.

Recall: of the real problems, how many it reported. Noise: of what it reported, how much was wrong. Raising recall is easy. Noise is what it costs the author.

skills/review-eval/SKILL.md.

ACT VI · LEARNING

A rejected comment keeps your reason. It is not fixed twice.

And a change to the lenses is kept only if the eval says it helped.

ACT VI · LEARNING

It pays off after a few rounds. With one or two comments there is nothing to learn from yet.

Start with review-loop threads sync.

VII

For managers

What to decide, what to measure, how to start.

The question is not "did a human read it?" It is what could go wrong, and which check should catch it.

The post calls reading by a person an imprecise proxy for that question.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

ACT VII · FOR MANAGERS

Who does what

Machines read

- Routine mistakes in a diff
- Mechanical patterns, as sweeps
- Whether the tests ran and passed

People keep

- Whether to build it at all
- Risky code: billing, the updater, CI workflows
- Architecture
- Owning the outcome

The split is from Osmani, Lunt and Wang, 15 October 2026.

ACT VII · FOR MANAGERS

Whoever prompted the change is on the hook for it. An agent's approval never counts as a person's.

The gate checks that a review was written. It never replaces the person who signs off.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

ACT VII · FOR MANAGERS

What to measure

MEASURE	WHERE IT COMES FROM
Human comments, and the time a PR waits on a person	Your tracker. review-loop does not measure it
Recall and noise of the review	`review-loop eval report`
Outside findings per round: the number should fall	`review-loop score show`
Reverts and incidents: the guardrail	Your data. Turn each into an eval case

The measures are from Osmani, Lunt and Wang, 15 October 2026. The right column is review-loop's README.

ACT VII · FOR MANAGERS

The cost argument is human time, not tokens.

A senior engineer pulled out of deep work to catch a routine mistake costs far more than an automated review. The post gives no dollar or token figures, and neither does this talk. review-loop records the tokens each run used, so you can measure your own.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

ACT VII · FOR MANAGERS

A way to roll it out

Week 1

One repository, or one or two developers with the files kept privately. Levels 1 and 2.

Weeks 2 to 4

Tune REVIEW.md as reviewers disagree with it.

Then

Level 3: the gate, for the paths that guard something.

After a few rounds

Level 4: learn from outside review.

The post: start with one repository, tune REVIEW.md for a few weeks, then widen. The mapping to levels is ours.

ACT VII · FOR MANAGERS

Start strict, loosen with evidence.

In review-loop a guard path always gets three readers. Automatic risk tiers from the paths a change touches are planned, not built.

Osmani, Lunt and Wang, "How the Claude Code team does code review", 15 October 2026.

ACT VII · FOR MANAGERS

Four questions for your team

Ownership

Who owns a change an agent wrote?

Guards

Which paths always need a person?

Measures

What would make us tighten review?

Memory

Where do review findings go after the round?

VIII

Limits

What it does not do.

ACT VIII · LIMITS

A small, generic version of some of these pieces

It does

- Lenses, sweeps and verification
- Several readers and a vote
- A report, a gate and a score
- Retro and eval

It does not

- Merge queues and staged rollout
- Feature flags
- Risk-tiered approval (planned)
- Explainers for modules (planned)

README, Background, and PLAN.md.

ACT VIII · LIMITS

Three readers cost about three times one. Say so before launching.

The tool asks first for a large diff, suggests a model per reader, and the eval records the tokens each run used.

ACT VIII · CLOSE

On Monday

```
review-loop depth main HEAD
review-loop sweeps main HEAD
/pre-pr-review
```

Free, free, and one reader. Fix what it finds. Stop here if that is enough.

Levels 1 and 2 of the README.

ACT VIII · CLOSE

Start at level 1. Stop when it is enough.

github.com/kenchris/review-loop

ACT VIII · CLOSE

Read more, and try it

The post

How the Claude Code team does code review
claude.dev, 15 October 2026

review-loop

The skills and scripts
github.com/kenchris/review-loop

This talk

The slides as a web page
kenchris.github.io/the-review-loop-site

IX

Hands-on

On your own branch. About 20 minutes.

HANDS-ON

Six steps

01

Install

review-loop doctor shows 8 ok.

02

Ask

/pre-pr-review on a branch with changes.

03

Compare

review-loop depth, then sweeps.

04

Setup

review-loop init, edit REVIEW.md.

05

Rule

Save a report, run the gate.

06

Learn

Optional: threads sync, then /retro.

HANDS-ON · STEPS 1 TO 3

Install, ask, compare

```
./install.sh --link
review-loop doctor

/pre-pr-review

review-loop depth main HEAD
review-loop sweeps main HEAD
```

Did the review find something the sweeps did not? Did the sweeps ask something the review answered?

HANDS-ON · STEPS 4 AND 5

Setup, then the rule

```
review-loop init
review-loop doctor

/pre-pr-review multipass my-branch
review-loop gate main HEAD
```

Edit REVIEW.md first: three lines on what failure means in your project.

HANDS-ON · STEP 6

For those with open pull requests and review comments.

review-loop threads sync --repo owner/name, then decide each thread.
Then /retro, and read the list before you apply anything.